Familiarización con Python

Fase	Semana	Día	Lección
5 - Python	1	1	2

¿Qué es una variable?

Una variable es un contenedor que se utiliza para almacenar datos que pueden cambiar durante la ejecución de un programa.

Las variables tienen un contexto y un alcance.

El contexto se refiere al lugar donde se define y se usa una variable en el código. Puede ser global, local o nonlocal.

El alcance determina la visibilidad y accesibilidad de una variable en diferentes partes del código. Puede ser global o local.

```
# Este es un comentario de una línea en Python
variable = 10  # Asignación de valor a una variable
```

¿Qué es una función?

Una función es un bloque de código reutilizable que realiza una tarea específica.

Puede recibir datos como parámetros y puede devolver un resultado se crea con la palabra reservada def.

```python
def suma(a: int, b: int) -> int:
    # Cuerpo de la función
    resultado = a + b
    return resultado
```

```python
def saludar() -> None:
    """
    Función para saludar
    :return:
    """
    print("Hola, ¡soy una función!")
```

¿Por qué comentar código?

Los comentarios son texto explicativo dentro del código fuente.

Mejoran la comprensión del código para los desarrolladores en el futuro (inclusive tu mismo)

No afectan la ejecución del programa, es decir se ignoran durante la ejecución.

```
// Querido colega programador:
//
// Cuando escribí este código, sólo Dios y yo
// sabíamos cómo funcionaba.
// Ahora, ¡sólo Dios lo sabe!
//
// Así que si está tratando de 'optimizarlo'
// y fracasa (seguramente), por favor,
// incremente el contador a continuación
// como una advertencia para su siguiente colega:
//
// total_horas_perdidas_aquí = 189
```

Tipos de comentarios

Comentarios de una línea

```
# Este es un comentario de una línea en Python
variable = 10  # Asignación de valor a una variable
```

Comentarios de múltiples líneas

```
"""
Este es un comentario
de múltiples líneas en Python.

Puede extenderse en varias líneas.
"""
```

Tipos de datos básicos

- Enteros (int)
  - Representan números enteros positivos o negativos
  - Ejemplo: x = 5
- Flotantes (float)
  - Representan números decimales.
  - Ejemplo: y = 3.14
- Booleanos (bool)
  - Representan valores de verdad: Verdadero o Falso.
  - Ejemplo: es_verdadero = True
- Cadenas de texto (str)
  - Representan texto.
  - Ejemplo: texto = "Hola, mundo!"

```
x = 5
y = 3.14
verdadero = True
falso = False
texto = "Hola, mundo!"
```

Tipos de datos estructurados

- Listas (list)
  - Colección ordenada y modificable de elementos.
  - Ejemplo: mi_lista = [1, 2, 3, 4]
- Tuplas (tuple)
  - Colección ordenada e inmutable de elementos.
  - Ejemplo: mi_tupla = (10, 20, 30)
- Diccionarios (dict)
  - Colección no ordenada de pares clave-valor.
  - Ejemplo: diccionario = {"nombre": "Juan", "edad": 30}
- Conjuntos (set)
  - Colección no ordenada y sin elementos duplicados.
  - Ejemplo: mi_conjunto = {1, 2, 3, 4}

```
mi_lista = [1, 2, 3, 4]
mi_tupla = (10, 20, 30)
diccionario = {
    "nombre": "Liz",
    "edad": 30
}
mi_conjunto = {1, 2, 3, 4}
```

Diferencias entre Duplas y Listas

Listas

- Lista Mutable: Puedes modificar, agregar o eliminar elementos después de crearla.
- Sintaxis: Se define usando corchetes [].
- Uso: Se usa cuando necesitas una colección que pueda cambiar en el tiempo.

Tupla

Inmutable: No puedes cambiar los elementos después de su creación. Una vez creada, no puede ser modificada.
Sintaxis: Se define usando paréntesis ().
Uso: Se usa cuando necesitas asegurarte de que los datos no cambien o cuando se necesita una colección de elementos constantes.

```
mi_lista = [1, 2, 3, 4]
mi_tupla = (10, 20, 30)
```

Conversión entre tipos de forma explícita

- De entero a flotante:
  - entero = 5
  - flotante = float(entero)
- De flotante a entero:
  - flotante = 3.14
  - entero = int(flotante)
- De cadena a entero o flotante:
  - cadena_entero = "10"
  - cadena_flotante = "3.14"
  - entero_desde_cadena = int(cadena_entero)
  - flotante_desde_cadena = float(cadena_flotante)

```
entero = 5
flotante = float(entero)
# Resultado: 5.0

flotante2 = 3.14
entero = int(flotante2)
# Resultado: 3

cadena_entero = "10"
cadena_flotante = "3.14"

entero_desde_cadena = int(cadena_entero)
# Resultado: 10
flotante_desde_cadena = float(cadena_flotante)
# Resultado: 3.14
```

Conversión entre tipos de forma implícita

- Es importante destacar este comportamiento para prevenir resultados no deseados
- De entero a flotante

```
>>> entero = 5
... pi = 3.14
...
... # Conversión automática
... resultado = entero + pi
>>> resultado
8.14
```

Interpolación de texto

- Podemos también formatear texto con valores que necesitamos de variables previas

```
>>> # Variables para el cálculo del área del círculo
... radio = 5
... pi = 3.14159
...
... # Combinación de texto y variables en una cadena
... resultado = f"El área de un círculo con radio {radio} es: {pi * radio ** 2}"
...
>>> resultado
'El área de un círculo con radio 5 es: 78.53975'
```

Operadores Aritméticos

Suma

```
>>> a = 5 + 3
>>> a
8
```

Resta

```
>>> b = 7 - 2
>>> b
5
```

Multiplicación

```
>>> c = 4 * 6
>>> c
24
```

Operadores Aritméticos

División

```
>>> d = 10 / 3
>>> d
3.3333333333333335
```

División entera

```
>>> e = 10 // 3
>>> e
3
```

Exponenciación

```
>>> f = 2 ** 3
>>> f
8
```

Módulo

```
>>> g = 10 % 3
>>> g
1
```

Operadores de Comparación

Igualdad(==)

```
>>> 5 == 5
True
```

Desigualdad(!=)

```
>>> 5 != 3
True
```

Mayor (>)

```
>>> 7 > 3
True
```

Menor (<)

```
>>> 4 < 2
False
```

Mayor o igual (>=)

```
>>> 5 >= 5
True
```

Menor o igual (<=)

```
>>> 6 <= 3
False
```

Operadores de asignación

Asignación (=)

```
>>> x = 5  # Asigna el valor 5 a la variable x
>>> x
5
```

Asignación con Operación:

```
>>> y = 10
... y += 5  # Equivalente a: y = y + 5
>>> y
15
```

Operadores Lógicos

- Operador and
  - Devuelve True si ambas condiciones son True.

```
>>> True and True
True
>>> True and False
False
```

- Operador or
  - Devuelve True si al menos una condición es True.

```
>>> True or True
True
>>> True or False
True
```

- Operador not
  - Niega el valor de la condición.

```
>>> not True
False
>>> not False
True
```

Captura de entrada y salida en la consola

Función input()

```
>>> nombre = input("Ingrese su nombre: ")
Ingrese su nombre: >? Luis
>>> nombre
'Luis'
```

Función print()

```
>>> print("Hola,", nombre)
Hola, Luis
```

Ejercicio 1

1. Cree una función de bienvenida para un usuario que se encargue de dar la bienvenida a un usuario X debe solicitar el nombre del usuario
2. Cree una funcion que se encargue de tomar dos notas y determine su promedio
3. Cree una funcion que permita mostrar el resultado del promedio al usuario, se debe mostrar el nombre del usuario y el promedio

Ejercicio 2

1. Dada una edad clasifique esa edad si es nino, adolecente, adulto o adulto mayor
2. Se debe de crear una funcion para cada una de las validaciones
3. Imprimir al final el resultadode la validacion

Condiciones

edad < 13 = niño
edad > = 13 && edad < 18 = Adolecente
edad > = 18 && edad < 60 = Adulto
edad > =60 = Adulto Mayor

Ejercicio 3

Escribe un programa en Python que cuente cuántas vocales hay en una frase ingresada por el usuario.

Ejercicio 4

Escribe un programa que solicite al usuario una lista de números y luego cuente cuántos de esos números son pares y cuántos son impares.

- Pide al usuario que ingrese una lista de números separados por comas.
- Convierte la entrada en una lista de enteros.
- Recorre la lista y cuenta cuántos números son pares y cuántos son impares.
- Muestra el resultado al usuario.